

PERTEMUAN 6

VIEWING 3D

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu memahami dan mengimplementasikan representasi data grafis objek 3D.

Pokok Bahasan:

1. Pengertian objek 3D
2. Sistem Koordinat 3D
3. Objek 3D
4. Struktur data objek 3D
5. Alur proses rendering grafis objek 3D
6. Implementasi objek primitive 3D.
7. Soal Latihan

6.1 Pengertian objek 3D

Grafik komputer 3 dimensi biasa disebut 3D atau adalah bentuk dari benda yang memiliki panjang, lebar, dan tinggi. Grafik 3 Dimensi merupakan teknik penggambaran yg berpatokan pada titik koordinat sumbu x(datar), sumbu y(tegak), dan sumbu z(miring).

Representasi dari data geometrik 3 dimensi sebagai hasil dari pemrosesan dan pemberian efek cahaya terhadap grafika komputer 2D. Tiga Dimensi, biasanya digunakan dalam penanganan grafis. 3D secara umum merujuk pada kemampuan dari sebuah video card (link). Saat ini video card menggunakan variasi dari instruksi-instruksi yang ditanamkan dalam video card itu sendiri (bukan berasal dari software) untuk mencapai hasil grafik yang lebih realistis dalam memainkan game komputer.

Grafik 3D merupakan perkembangan dari grafik 2D. Didalam grafika komputer, 3D merupakan bentuk grafik yang menggunakan representasi data geometri tiga dimensi. Perbedaan paling utama dari objek 2 dimensi dan objek 3 dimensi yaitu kedalamannya. Yaitu suatu jarak antara pengamat dengan suatu objek yang dilihat (point of view). Jadi, jika objek 2 dimensi hanya menggunakan 2 ukuran (panjang dan lebar), maka 3 dimensi menggunakan 3 ukuran meliputi panjang, lebar, kedalaman yang disimbolkan dengan sumbu x, y, z.

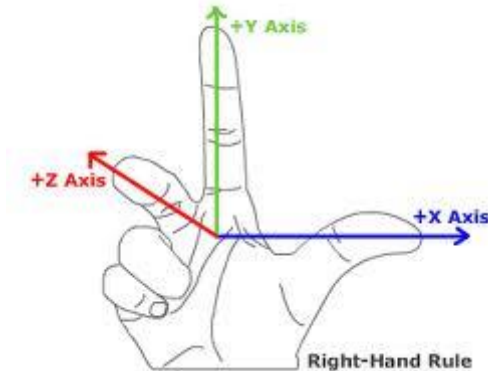
Suatu objek rangka 3D apabila disinari dari arah tertentu akan membentuk bayangan pada permukaan gambar. Proses pembuatan grafik komputer 3D dapat dibagi ke dalam tiga fase, yaitu 3D modeling yang mendeskripsikan bentuk dari sebuah objek, layout dan animation yang mendeskripsikan gerakan dan tata letak sebuah objek, dan 3D rendering yang memproduksi image dari objek tersebut.

Di dalam dunia game, 3D secara umum merujuk pada kemampuan dari sebuah video card (link). Saat ini video card menggunakan variasi dari instruksi-instruksi yang ditanamkan dalam video card itu sendiri (bukan berasal dari software) untuk mencapai hasil grafik yang lebih realistis dalam memainkan game komputer.

6.2 Sistem Koordinat 3D

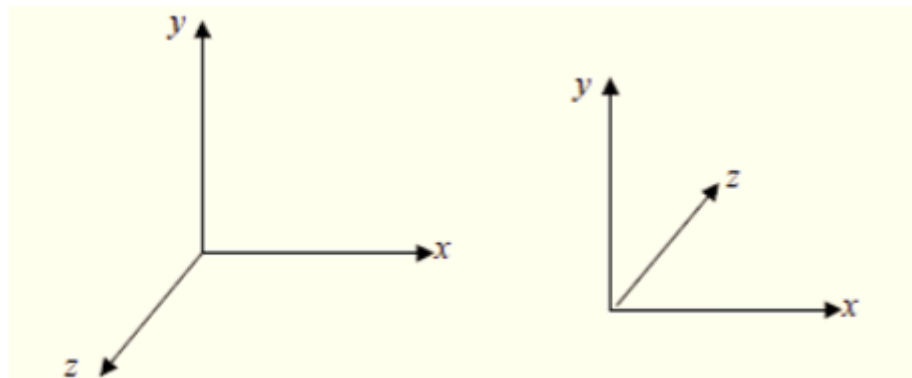
Secara umum, sistem koordinat tiga dimensi dibedakan menjadi dua, yaitu sistem koordinat tangan kiri dan sistem koordinat tangan kanan. Sistem koordinat tangan kanan dapat digambarkan dengan tiga buah jari pada tangan kanan yang menghadap pengamat yaitu jari tengah sebagai sumbu z positif, jari telunjuk sebagai sumbu y dan ibu jari sebagai sumbu x. Sedangkan sistem koordinat tangan kiri dapat digambarkan dengan tiga buah jari pada tangan

kiri dengan punggung tangan menghadap pengamat dengan jari tengah sebagai sumbu z negatif, jari telunjuk sebagai sumbu y dan ibu jari sebagai sumbu x. Sistem koordinat **tangan kanan** banyak digunakan pada penggambaran **geometri**. Sebaliknya sistem koordinat **tangan kiri** banyak digunakan pada penggambaran pada **grafika komputer**. Sistem koordinat tangan kiri digunakan agar objek 3 dimensi memiliki nilai z positif (jarak jauh dekat benda lebih terlihat).



Gambar 6.1 Kaidah Tangan Kanan Sumbu Koordinat 3 Dimensi

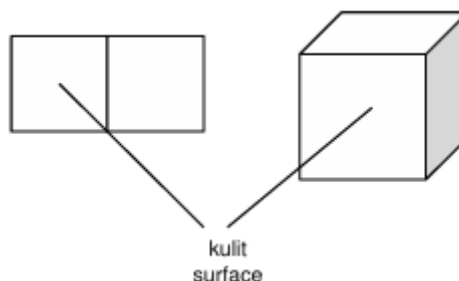
Setiap titik dalam sistem koordinat tiga dimensi dapat ditentukan posisinya dengan menentukan seberapa jauh, tinggi dan lebar dari objek aslinya yang ditentukan berdasarkan koordinat-koordinat pada sumbu x, y, dan z.



Gambar 6.2 Sumbu Koordinat 3 Dimensi

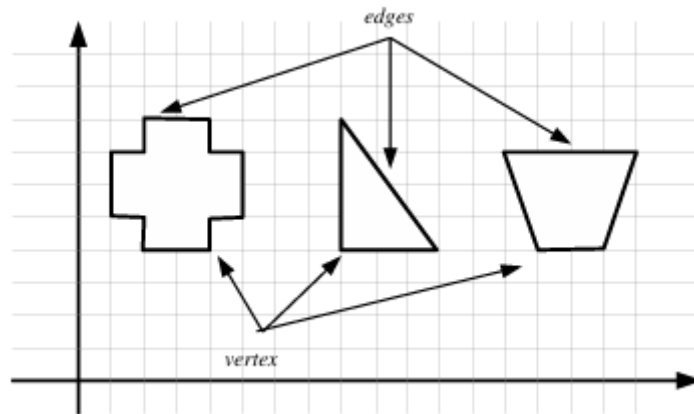
6.3 Objek 3D

Benda tiga dimensi disusun dari sekumpulan **surface**. Gambar di bawah memberikan contoh bagaimana surface digunakan untuk menyusun benda tiga dimensi.



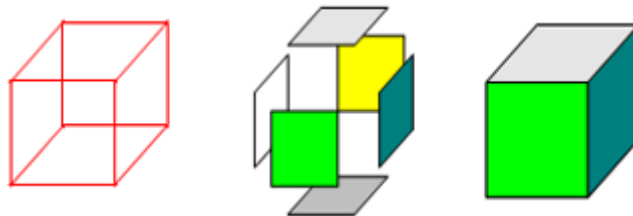
Gambar 6.3 Permukaan objek 3D

Surface dapat dibuat dari rangkaian **Polygon**. Polygon adalah **bentuk yang disusun dari serangkaian garis yang terhubung satu dengan yang lain dan berbentuk kurva tertutup sehingga membentuk sebuah objek gambar**. Titik sudut dari Polygon disebut **vertex** sedangkan garis penyusun Polygon disebut **edge**.



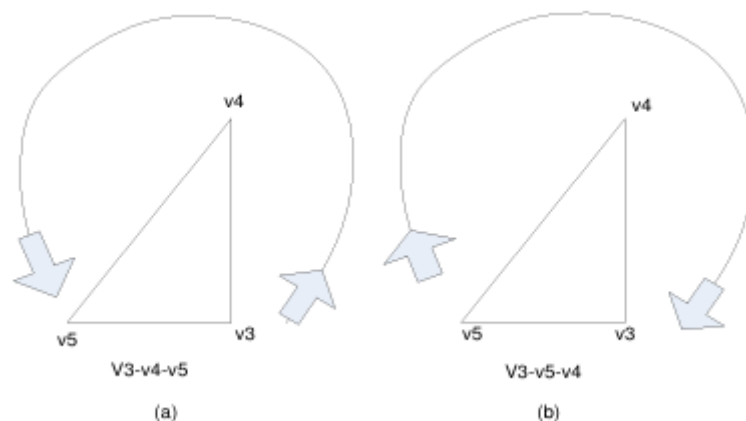
Gambar 6.4 Polygon

Polygon digambar dengan menggambar masing-masing **edge** dengan setiap **edge** merupakan pasangan dari **vertex_i – vertex_{i+1}** kecuali untuk **edge terakhir** merupakan pasangan dari **vertex_n – vertex₁**.



Gambar 6.5 Objek 3D yang tersusun dari kumpulan poligon

Bentuk polygon yang paling tepat digunakan untuk membuat permukaan benda tiga dimensi adalah **polygon segitiga (triangle)**. Hal ini disebabkan **polygon segitiga akan selalu berada dalam keadaan planar (datar)** sementara polygon dengan vertex lebih dari 3 dapat berada dalam kondisi non-planar karena salah satu vertex tidak berada di lokasi yang sejajar. Satu hal yang harus diperhatikan pada saat menggunakan polygon segitiga adalah **winding**. **Winding** adalah urutan dan arah vertex-vertex penyusun polygon. Winding menganggap bahwa arah vertex **berlawanan jarum jam sebagai tampak depan** sedangkan arah vertex **searah jarum jam dianggap sebagai bagian belakang benda**.



Gambar 6.6 arah vertex : (a) berlawanan jarum jam; (b) searah jarum jam

Pada gambar 6.6.a polygon menggunakan winding berlawanan jarum jam karena didefinisikan sebagai urutan vertex **v3-v4-v5** sedangkan pada gambar 6.6.b menggunakan winding searah jarum jam karena urutan vertex yang digunakan **v3-v5-v4**.

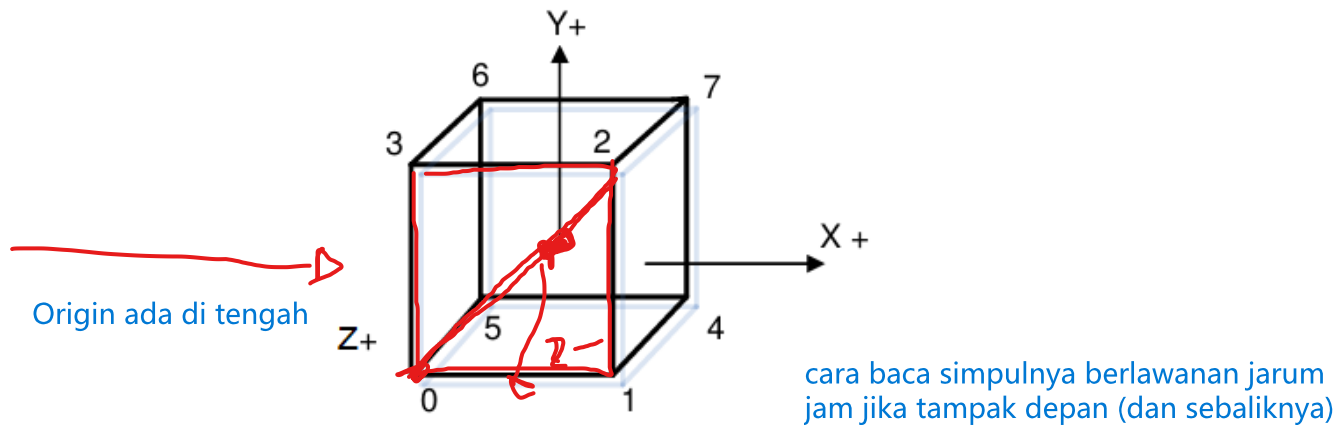
Pada setiap permukaan dari poligon yang membentuk objek akan memunculkan garis Normal, yaitu **vektor tegak lurus pada permukaan**. Fungsinya untuk bayangan benda tersebut.



Gambar 6.7 Garis normal poligon

6.4 Struktur data objek 3D

Sebuah benda tiga dimensi dapat disusun dengan menghubungkan sejumlah **vertex**. Kumpulan vertex yang saling terhubung tersebut disebut sebagai **wireframe** atau kerangka, sedangkan benda yang disusun dari kumpulan titik dan permukaan tersebut disebut sebagai **mesh**. Perhatikan Gambar 1.5, kubus yang ada di gambar tersebut dapat disusun berdasarkan data yang ada pada tabel 6.1 dan tabel 6.2



Gambar 6.8 Kubus

Tabel 6.1 vertex penyusun kubus

Vertex	X	Y	Z
0	-1	-1	+1
1	+1	-1	+1
2	+1	+1	+1
3	-1	+1	+1
4	+1	-1	-1
5	-1	-1	-1
6	-1	+1	-1
7	+1	+1	-1

Tabel 6.2 Permukaan (face) kubus

Face	V1	V2	V3	keterangan
F1.a	0	1	2	Depan
F1.b	0	2	3	Depan
F2.a	1	4	7	Kanan
F2.b	1	7	2	Kanan
F3.a	4	5	6	Belakang

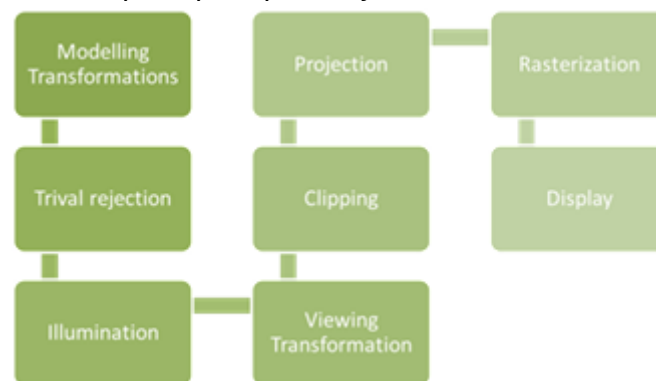
$$2^3 = 8$$

Face	V1	V2	V3	keterangan
F3.b	4	6	7	Belakang
F4.a	5	3	0	Kiri
F4.b	5	6	3	Kiri
F5.a	3	2	7	Atas
F5.b	3	7	6	Atas
F6.a	0	1	4	Bawah
F6.b	0	4	5	Bawah

Tabel 6.2 Menyatakan urutan vertex yang menyusun sebuah permukaan. Untuk setiap permukaan didefinisikan sebagai polygon segitiga (triangle), sebagai contoh permukaan kubus bagian depan didefinisikan melalui dua buah segitiga, yaitu F1.a dan F1.b yang masing-masing disusun dari vertex 0, vertex 1, dan vertex 2, serta vertex 0, vertex 2 dan vertex 3.

6.5 Alur proses rendering grafis objek 3D

Pada grafika komputer, rendering adalah proses pembuatan/ produksi image/gambar pada layar dari pendeskripsian suatu model. Proses rendering grafis 3D mendeskripsikan objek 3D dari vertex-vertex pada objek primitif (seperti segitiga, titik, garis, dan segiempat), dan menghasilkan warna-warna pada pixel pada layar.



Gambar 6.9 Alur proses rendering grafis 3D

1. **Modelling Transformations** : Menempatkan model pada sistem koordinat yang disebut world space. Semua objek, sumber cahaya, dan sudut pandang pengamat berada pada world space.
2. **Trivial Rejection** : Rejection (penolakan) dilakukan terhadap bagian-bagian yang tidak perlu sebagai upaya optimalisasi.
3. **Illumination** : Cahaya menyebabkan suatu objek dapat terlihat. Warna objek ditentukan dari properti material objek itu sendiri dan sumber cahaya pada tampilan tersebut. Algoritma iluminasi tergantung pada pemodelan bayangan (shading model) dan pemodelan permukaan (surface model).
4. **Viewing transformation** : Transformasi pengamatan adalah bentuk lain dari koordinat sistem. Memetakan titik-titik yang ada pada world space ke ruang pengamatan. Posisi pengamat ditransformasikan ke asalnya. Arah pengamatan diorientasikan sepanjang axis. Mendefinisikan pengamatan volume
5. **Clipping** : Proses clipping dilakukan pada penggambaran volume 3D yang biasa disebut viewing frustum. Pada tahap ini dilakukan eliminasi total pada objek (dan bagian objek) yang tidak visible bagi citra.

6. Projection : Objek selanjutnya diproyeksikan ke dalam 2D. Transformasikan dari ruang pengamatan ke layar
7. Rasterization : Konversi tiap objek primitive ke suatu set fragment. Sebuah fragment diperlakukan seperti pixel pada ruang 3 dimensi, selaras dengan grid pixel, dengan atribut seperti posisi, warna, dan tekstur yang normal.
8. Display : Melakukan interpolasi parameter. Melakukan beragam operasi 2D.

6.6 Implementasi objek primitive 3D

Ada beberapa bentuk primitive dari objek 3 dimensi. Primitive yaitu bentuk dasar yang dapat dengan mudah didefinisikan secara matematik. Setiap primitive 3D memiliki parameter nilai yang mengekspresikan letak dan bentuk primitive objek tersebut.

A. Sphere (bola)

```
int detail = 30; // Higher = smoother sphere
float radius = 100;

void setup() {
  size(400, 400, P3D);
  noFill();
  stroke(0, 255, 200); // Wireframe color
}

void draw() {
  background(20);
  lights();

  translate(width / 2, height / 2, 0);
  rotateY(frameCount * 0.01);
  rotateX(frameCount * 0.005);

  drawWireframeSphere(radius, detail);
}

void drawWireframeSphere(float r, int detail) {
  // Latitude (horizontal rings)
  for (int i = 0; i <= detail; i++) {
    float theta1 = map(i, 0, detail, -HALF_PI, HALF_PI);
    float y = r * sin(theta1);
    float ringRadius = r * cos(theta1);

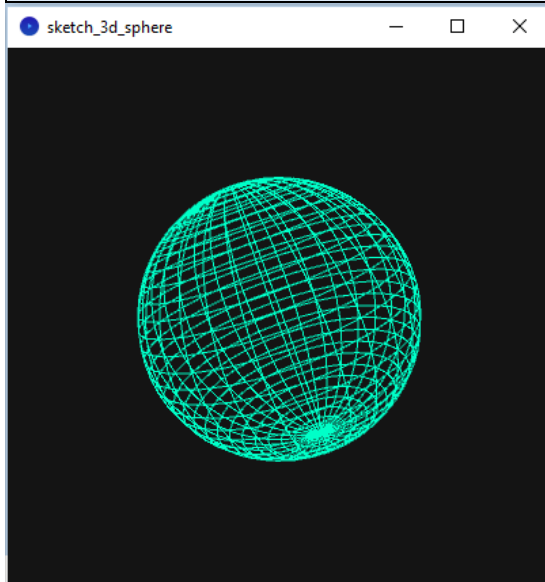
    beginShape();
    for (int j = 0; j <= detail; j++) {
      float theta2 = map(j, 0, detail, 0, TWO_PI);
      float x = ringRadius * cos(theta2);
      float z = ringRadius * sin(theta2);
      vertex(x, y, z);
    }
    endShape();
  }

  // Longitude (vertical lines)
  for (int j = 0; j <= detail; j++) {
    float theta2 = map(j, 0, detail, 0, TWO_PI);
    beginShape();
    for (int i = 0; i <= detail; i++) {
      float theta1 = map(i, 0, detail, -HALF_PI, HALF_PI);
      float y = r * sin(theta1);
      float ringRadius = r * cos(theta1);
```

```

    float x = ringRadius * cos(theta2);
    float z = ringRadius * sin(theta2);
    vertex(x, y, z);
  }
  endShape();
}
}

```



B. Cube (kubus)

```

float cubeSize = 150;

void setup() {
  size(400, 400, P3D);
  noFill();
  stroke(255, 100, 0); // Wireframe color
}

void draw() {
  background(30);
  lights();

  translate(width / 2, height / 2, 0);
  rotateX(frameCount * 0.01);
  rotateY(frameCount * 0.01);

  drawWireframeCube(cubeSize);
}

void drawWireframeCube(float s) {
  float h = s / 2;

  PVector[] vertices = new PVector[8];

  // Define 8 corners of the cube
  vertices[0] = new PVector(-h, -h, -h);
  vertices[1] = new PVector( h, -h, -h);
  vertices[2] = new PVector( h,  h, -h);
  vertices[3] = new PVector(-h,  h, -h);
  vertices[4] = new PVector(-h, -h,  h);
  vertices[5] = new PVector( h, -h,  h);

```

```

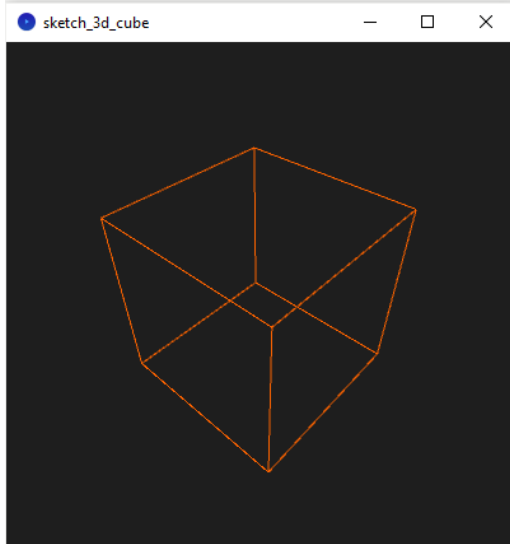
vertices[6] = new PVector( h,  h,  h);
vertices[7] = new PVector(-h,  h,  h);

// Draw edges
line(vertices[0].x,  vertices[0].y,  vertices[0].z,  vertices[1].x,
vertices[1].y,  vertices[1].z);
line(vertices[1].x,  vertices[1].y,  vertices[1].z,  vertices[2].x,
vertices[2].y,  vertices[2].z);
line(vertices[2].x,  vertices[2].y,  vertices[2].z,  vertices[3].x,
vertices[3].y,  vertices[3].z);
line(vertices[3].x,  vertices[3].y,  vertices[3].z,  vertices[0].x,
vertices[0].y,  vertices[0].z);

line(vertices[4].x,  vertices[4].y,  vertices[4].z,  vertices[5].x,
vertices[5].y,  vertices[5].z);
line(vertices[5].x,  vertices[5].y,  vertices[5].z,  vertices[6].x,
vertices[6].y,  vertices[6].z);
line(vertices[6].x,  vertices[6].y,  vertices[6].z,  vertices[7].x,
vertices[7].y,  vertices[7].z);
line(vertices[7].x,  vertices[7].y,  vertices[7].z,  vertices[4].x,
vertices[4].y,  vertices[4].z);

line(vertices[0].x,  vertices[0].y,  vertices[0].z,  vertices[4].x,
vertices[4].y,  vertices[4].z);
line(vertices[1].x,  vertices[1].y,  vertices[1].z,  vertices[5].x,
vertices[5].y,  vertices[5].z);
line(vertices[2].x,  vertices[2].y,  vertices[2].z,  vertices[6].x,
vertices[6].y,  vertices[6].z);
line(vertices[3].x,  vertices[3].y,  vertices[3].z,  vertices[7].x,
vertices[7].y,  vertices[7].z);
}

```



C. Cone (kerucut)

```

int sides = 30;           // Number of segments around the base (higher =
smoother)
float radius = 100;       // Radius of the base
float height = 200;       // Height of the cone

void setup() {
  size(600, 600, P3D);
  noFill();
}

```



```

    stroke(0, 255, 100); // Wireframe color
}

void draw() {
    background(20);
    lights();

    translate(width / 2, height / 2, 0);
    rotateY(frameCount * 0.01);
    rotateX(frameCount * 0.005);

    drawWireframeCone(radius, height, sides);
}

void drawWireframeCone(float r, float h, int segments) {
    PVector[] basePoints = new PVector[segments];

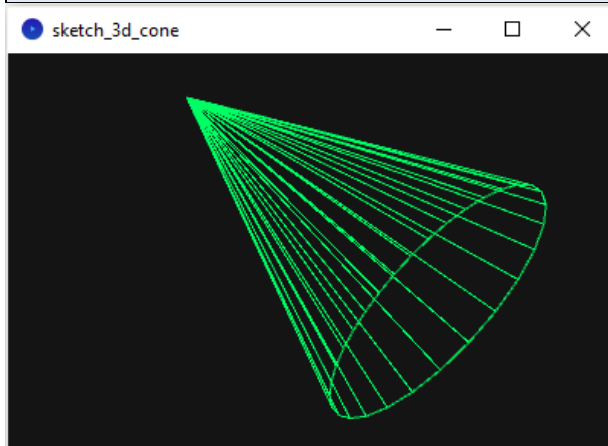
    // Create circle points for the base
    for (int i = 0; i < segments; i++) {
        float angle = TWO_PI / segments * i;
        float x = cos(angle) * r;
        float z = sin(angle) * r;
        basePoints[i] = new PVector(x, h / 2, z);
    }

    PVector apex = new PVector(0, -h / 2, 0); // Top point of the cone

    // Draw base circle
    beginShape();
    for (int i = 0; i < segments; i++) {
        vertex(basePoints[i].x, basePoints[i].y, basePoints[i].z);
    }
    endShape(CLOSE);

    // Draw lines from base to apex
    for (int i = 0; i < segments; i++) {
        line(basePoints[i].x, basePoints[i].y, basePoints[i].z,
            apex.x, apex.y, apex.z);
    }
}

```



D. Torus (cincin)

```

int sides = 40;      // Detail around the tube
int rings = 40;      // Detail around the torus ring
float r = 80;        // Minor radius (tube radius)
float R = 200;       // Major radius (distance from center to tube)

void setup() {
  size(800, 800, P3D);
  stroke(255, 0, 0);
  noFill(); // <- comment this if you want solid surface
}

void draw() {
  background(0);
  lights();

  translate(width/2, height/2, 0);
  rotateX(frameCount * 0.01);
  rotateY(frameCount * 0.01);

  drawTorus(R, r, rings, sides);
}

void drawTorus(float R, float r, int rings, int sides) {
  for (int i = 0; i < rings; i++) {
    float theta1 = TWO_PI * i / rings;
    float theta2 = TWO_PI * (i + 1) / rings;

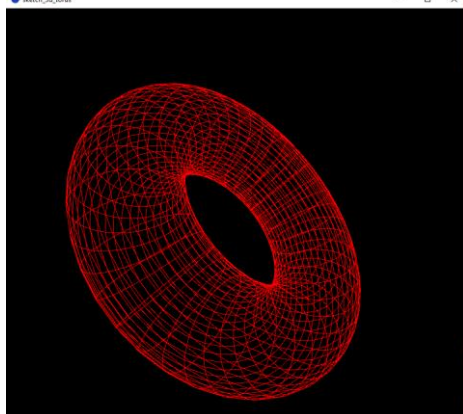
    beginShape(QUAD_STRIP);
    for (int j = 0; j <= sides; j++) {
      float phi = TWO_PI * j / sides;

      float x1 = (R + r * cos(phi)) * cos(theta1);
      float y1 = (R + r * cos(phi)) * sin(theta1);
      float z1 = r * sin(phi);

      float x2 = (R + r * cos(phi)) * cos(theta2);
      float y2 = (R + r * cos(phi)) * sin(theta2);
      float z2 = r * sin(phi);

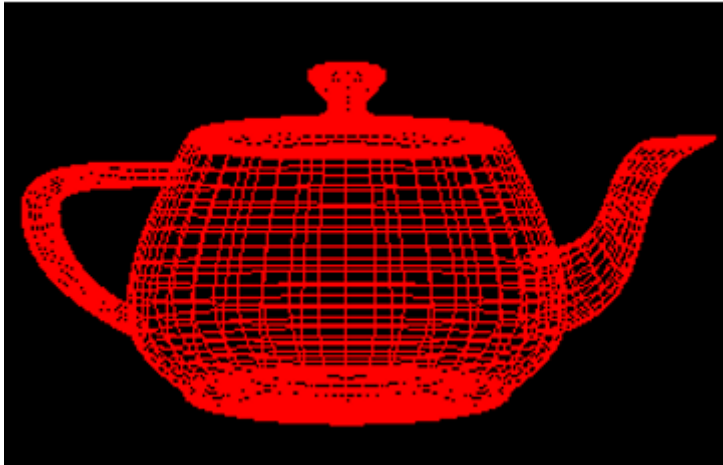
      vertex(x1, y1, z1);
      vertex(x2, y2, z2);
    }
    endShape();
  }
}

```



6.7 Soal Latihan

1. Buatlah sintaks untuk menampilkan output 3d objek seperti gambar berikut!



Daftar Pustaka

1. Andono, Pulung Nurtantio., Sutojo, T. 2016. Konsep Grafika Komputer. Yogyakarta: CV Andi Offset.
2. David F. Rogers, Alan J. Adams, Mathematical Elements for Computer Graphics (2nd edition), McGraw-Hill, 1989.
3. Edward Angel. 2005. *Interactive Computer Graphics: A Top-Down Approach with OpenGL 2nd*. Addison Wesley.
4. Grafika Komputer untuk Mahasiswa dan Umum. Universitas Negeri Malang.
5. John F. Hughes, Andries Van Dam, Morgan Mcguire, David F. Sklar, James D. Foley, Steven K. Feiner, Kurt Akeley. 2014. *Computer Graphics: Principles and Practice (3rd edition)*. Addison-Wesley.
6. Klawonn, Frank. 2012. *Introduction to Computer Graphics Using Java 2D and 3D 2nd Edition*. Springer-Verlag: London.
7. Maliki, Irfan. 2006. Grafika Komputer. Jakarta Selatan: Mediatek.
8. Siahaan, Vivian., Sianipar, Rismon H., Java untuk Grafika Komputer dan Animasi, SPARTA, 2018.